

H1. 시계열과 파이썬

서울 기온 데이터로 시계열을 코드로 다루기 시작한다

시계열 분석 8모듈 지도

1. H1. 시계열과 파이썬 ← 오늘
2. H2. 분포로 보는 시계열
3. H3. 평활 — 이동평균과 지수평활
4. H4. 계절성과 분해
5. H5. 자기상관
6. H6. 예측과 평가
7. H7. 회귀로 예측
8. H8. 미니 프로젝트

서사 축: 본다 → 분포로 잔다 → 매끈하게 편다 → 쪼갬다 → 시간의 관계를 잔다 → 예측하고 평가한다 → 내 데이터로.

오늘의 목표

- 시계열의 정의(순서가 정보)를 안다.
- Colab에서 pandas로 CSV를 읽어 날짜 인덱스를 세울 수 있다.
- 그래프를 그리고, 날짜로 데이터를 조회할 수 있다.

시계열이란 — 순서가 정보다

- 숫자를 시간 순서대로 기록한 데이터를 시계열(time series)이라 한다.
- 핵심은 순서 자체가 정보라는 것 — 순서를 섞으면 "자라는 중인가?", "오르는 중인가?" 같은 질문에 답할 수 없게 된다.
- 확률과 통계에서 다루는 표본은 대부분 순서가 없다. 시계열은 순서가 있는 데이터를 다루는 분야다.

시계열인 것 / 아닌 것

- 시계열: 편의점의 하루 매출을 한 달 동안 기록, 놀이공원의 월별 방문자 수.
- 시계열 아님: 학급 학생들의 혈액형 분포, 교실 사물함 번호.
- 판별 기준: 시간(순서)을 섞어도 데이터의 의미가 그대로인가?

Colab이란

- 설치 없이 브라우저에서 쓰는 파이썬 공책(노트북).
- 구글 계정만 있으면 바로 쓸 수 있다.
- colab.research.google.com에 접속해 새 노트북을 만든다.
- 빈 칸(셀)에 코드를 쓰고 **Shift+Enter**로 실행한다. 노트북은 위에서부터 순서대로 실행하는 것이 규칙이다.

첫 셀

```
print("시계열 시작")
```

파이썬은 계산기이기도 하다.

```
(24.8 + 25.2 + 24.3) / 3
```

3년치 데이터 읽기 — 읽는 법

```
pd.read_csv(url, parse_dates=["date"], index_col="date")
```

= url의 CSV를 읽되 / date 열을 날짜로 인식시키고 /
그 날짜를 각 행의 이름표(인덱스)로 삼는다

- `import pandas as pd` — 표 데이터 전용 도구 상자를 꺼내 `pd` 라는 별명으로 부른다.
- `parse_dates=["date"]` — 컴퓨터는 "2024-01-01" 같은 값을 그냥 글자로 읽는다. 이 옵션이 그것을 진짜 날짜로 바꾼다.
- `index_col="date"` — 날짜를 행의 이름표로. 잠시 뒤 "행 번호" 대신 날짜로 데이터를 부르기 위한 준비다.

데이터 읽기 코드

```
import pandas as pd

url = "https://raw.githubusercontent.com/jaehoonkim/timeseries-academy/main/data/seoul-temp-daily.csv"
df = pd.read_csv(url, parse_dates=["date"], index_col="date")
df.head()
```

확인해보기 — len과 tail

```
len(df)
```

```
df.tail()
```

`len(df)` 는 표의 행 수, 즉 데이터가 며칠치인지 알려준다. `tail()` 은 표의 끝부분 몇 줄을 보여준다 — 결과는 활동지에서 직접 확인한다.

날짜 인덱스 — 이름표가 된 날짜

- 방금 `index_col="date"` 를 넣었으므로, `df` 의 행은 0, 1, 2... 번호가 아니라 날짜로 불린다.
- 이제 행을 번호가 아니라 날짜로 부를 수 있다 — 다음 도구가 `loc` 이다.

날짜로 부르기 — loc 읽는 법

```
df.loc["2025-08-01"]
```

= 이름표(인덱스)가 "2025-08-01"인 행을 찾아 달라

(loc = location)

- `df["temp_avg"]` 가 열(세로)을 고르는 것이라면, `df.loc[...]` 는 행(가로)을 고른다.
- "2026-01"처럼 달까지만 쓰면 그 달의 모든 행이 나온다.

loc으로 조회하기

```
df.loc["2025-08-01"]
```

```
df.loc["2026-01"]
```

```
df.loc["2026-01"]["temp_avg"].mean()
```

고른 결과에 함수를 이어 붙일 수도 있다 — 결과는 직접 실행해서 확인해 보자.

그래프 그리기

숫자를 눈으로 훑는 대신 그림으로 본다.

```
df["temp_avg"].plot()
```

```
df.plot()
```

`df["열이름"]` 이 한 열만 고르는 것이라면, `df` 를 통째로 넘기면 숫자 열 전부를 각각의 선으로 겹쳐 그린다.

'언제'를 묻는 법 — idxmax

```
df["temp_avg"].idxmax()
```

= temp_avg가 최대인 행의 이름표(=날짜) 를 알려 달라
(max() 가 "얼마나"라면 idxmax() 는 "언제")

```
df["temp_avg"].max()
```

```
df["temp_avg"].idxmax()
```

에러를 만나면

- `NameError: name 'df' is not defined` — 위 셀을 실행하지 않고 아래 셀부터 실행한 것. 노트북은 위에서부터 순서대로.
- `SyntaxError` — 따옴표.괄호 짝이 안 맞는 것. 에러 메시지의 화살표(^)가 위치를 가리켜 준다.
- 에러 메시지는 혼내는 게 아니라 **힌트**다.

오늘 배운 세 문장

1. 시계열은 순서가 정보다.
2. 날짜를 인덱스로 세우면, 데이터에 날짜로 질문할 수 있다.
3. `max` 는 "얼마나", `idxmax` 는 "언제".

이제 활동지로

1. Colab에서 첫 셀을 실행해 본다.
2. CSV를 불러와 `len` · `tail` 로 확인한다.
3. 그래프를 그리고, `loc` 으로 날짜를 조회한다.
4. 오늘 본 코드를 활동지에서 직접 실행하고, 나만의 관찰을 적어본다.